

WATER JUG

→ Lab 1. State space and combinatorial explosion.

State Space:-

A state space is the set of all possible configurations (states) a system can be in, along with the ways you can move between them (usually via "actions" or "transitions"). Think of it as a giant map:

- Nodes = distinct states of the system.
- Edges = legal moves/transitions from one state to another.

Combinatorial explosion

The problem is that state spaces tend to grow multiplicatively, not additively, as you add more variables or components. If each of n independent variables can take k values, the number of possible states is k^n .

- exponential growth.

- Chess has roughly 10^{43} to 10^{47} legal positions - vastly more than the number atoms on earth.
- Tic-tac-toe has only 9 cells with 3 possible values each → at most $3^9 = 19,683$ states (fewer once you account for legality), which is why computers solve it trivially.

Problem:-

We have two water jugs X , Y with capacities 4 litres (X) and 3 litres respectively. In the beginning both jugs are empty. Water can be used unlimited. Six operations performed on jugs are below:

1. Fill X → X becomes 4 (filled)
2. Fill Y → Y becomes 3 (filled 3 lit)
3. Empty X → X becomes 0
4. Empty Y → Y becomes 0

5. Pour $X \rightarrow Y \rightarrow$ transfer water from X to Y
until X empty or Y full

6. Pour $Y \rightarrow X \rightarrow$ transfer water from Y to X
until Y empty or X full.

Write a program that represents a state as (x, y) , writes a $\text{successors}(\text{state})$ function, expands the state space level from $(0,0)$, and prints how many new states each level adds.

Aim:

To represent the water jug problem as a state space, define the successor function for the problem, and implement a breadth-first search (BFS) that explores the state space level by level from the initial state printing the numbers of new states discovered at each level.

Problem Statement:

There are 2 jugs one with Jug A with capacity 4 litres and Jug B with capacity 3 litres. Neither Jug has markings, so, water can be ^{only} be measured by filling, emptying or pouring between the jugs. Starting with both jugs empty, the problem is to explore all the states reachable through the allowed operations.

State Representation:

A state is represented as an ordered pair (a, b)
where; a = current quantity of water in A (0 to 4 litres)
 b = current quantity of water in B (0 to 3 litres)

The initial state is $(0, 0)$ - both jugs empty.

Since 'a' can take 5 values $(0-4)$ and 'b' can take 4 values $(0-3)$, the total number of possible states is

$$\text{Total states} = (\text{CAP}_A + 1) \times (\text{CAP}_B + 1) = 5 \times 4 = 20$$

Successor Function (Rules of the Problem)

From any state (a, b) , the following six operations define all possible next states.

Successor(state) answers one question: from here,

what states can I reach in one move?

$x, y = \text{state}$ is Python tuple unpacking. It takes the pair $(4, 0)$ and puts 4 into x and 0 into y in one line.

#	Operation	Resulting state
1.	Fill Jug A	(CAP_A, b)
2.	Fill Jug B	(a, CAP_B)
3.	Empty Jug A	$(0, b)$
4.	Empty Jug B	$(a, 0)$
5.	Pour $A \rightarrow B$	$(a - \text{pour}, b + \text{pour})$,
6.	Pour $B \rightarrow A$	$(a + \text{pour}, b - \text{pour})$,

$$\text{ii) } \text{pour}_A = \min(a, \text{CAP}_B - b)$$

$$\text{vii) } \text{pour}_B = \min(b, \text{CAP}_A - a)$$

This gives a branching factor of 6 - every state has (upto) 6 successors.

def successors(state):

$x, y = \text{state}$

$$\text{pour_x_to_y} = \min(x, 3-y)$$

$$\text{pour_y_to_x} = \min(y, 4-x)$$

Workout how much water actually moves:

The four fill/empty operations are easy. The two pours need thought.

Pouring X into Y: X can give at most x litres (it has no more). Y can accept at most 3-y litres (that is its free space). The amount that moves is whichever is smaller - that is what min is doing.

return [(4, y),

1. Fill X

(x, 3),

2. Fill Y

(0, y),

3. Empty X

(x, 0),

4. Empty Y

(x - pour_x_to_y, y + pour_x_to_y), # 5. Pour X to Y

(x + pour_y_to_x, y - pour_y_to_x)] # 6. Pour Y to X

Return all six results

The function returns a plain list of six new states. Notice it does not check whether they are new, whether they are legal, or whether they are duplicates. That is deliberate. Successors has one job: say what the operations produce. Deciding what is new is a different job, done later. One function, one job.

start = (0, 0)

seen = {start} # Every state ever found (a set)

frontier = [start] # States found at the

level = 0 # Current level.

//_

Declare the four variables the search needs:

- seen is a set holding every state ever found. Sets test membership instantly and refuse duplicates.
- frontier is a list holding only the states found at the current level. This is what we expand next.
- level counts how deep we are. Both seen and frontier start containing just (0,0).

while frontier and level < 8;

new = []

Open the level loop:

while frontier and level < 8 has two stopping conditions: stop if there is nothing left to expand, and stop after 8 levels.

new = [] is emptied at the top of every level.

It will collect the states discovered at this level only

for s in frontier:

for ns in successors(s):

Two nested loops: every state, every move

The outer loop walks each state in the current frontier. The inner loop walks each of the six states that one move can reach.

Together they generate every state reachable in exactly one more move than the current level.

if ns not in seen:

seen.add(ns)

new.append(ns)

//_

The one line that stops the explosion:

if ns not in seen is the whole difference between a tree and a graph. Six moves from (0,0) produce four copies of (0,0) - you cannot empty an empty jug.

Seen.add(ns) must happen immediately, inside the if. If you added everything to seen after the loop finished, two states at the same level could both discover (1,3) and it would be counted twice.

"Delete this 'if' and the program explores 6, 36, 216, 1296... and never stops. Everything else in the program is bookkeeping; this line is the algorithm."

```
if ns[0] == 2;  
    print(" *** GOAL: 2L in X at level,  
          level+1, "***")
```

Check for the goal as soon as a state is created.
ns[0] is the first element of the tuple, which is Jug X. So ns[0] == 2 asks: does this state have 2 litres in X?

We print level+1 because level has not been incremented yet at this point in the loop.

```
level += 1
```

```
print("Level", level, "→", len(new), "new states", sorted(new))  
frontier = new
```


//_

Finish the level and move on

level $t+1$, then report, then frontier = new.

That last assignment is what makes this a level-by-level search: whatever we just discovered becomes what we expand next round. Everything found earlier is already in seen and will never be revisited.

```
print("Total states found:", len(seen))
```

Print the total and run it

len(seen) is the size of the entire reachable state space.

The program is complete. ~~Start~~.

LAB1: State space of the Water Jug problem

A state is (x, y) = litres in the 4L jug and the 3L jug.

```
def successors(state):
```

```
    x, y = state
```

```
    pour_x_to_y = min(x, 3-y) # x gives what it has,
```

```
    pour_y_to_x = min(y, 4-x) # y takes what fits
```

```
    return [(4, y),
```

```
            (x, 3),
```

```
            (0, y),
```

```
            (x, 0),
```

```
            (x - pour_x_to_y, y + pour_x_to_y),
```

```
            (x + pour_y_to_x, y - pour_y_to_x)]
```


//_

```
Start = (0,0)
Seen = {Start} # Every state ever found (< set)
frontier = [Start] # states found at the current level
level = 0
```

```
while frontier and level < 8:
```

```
    new = []
```

```
    for s in frontier:
```

```
        for ns in successors(s):
```

```
            if ns not in Seen:
```

```
                Seen.add(ns)
```

```
                new.append(ns)
```

```
            if ns[0] == 2:
```

```
                print(" *** GOAL: 2L in X 4  
                    level ", level+1, " ***")
```

```
        level += 1
```

```
        print("Level", level, "→", len(new), "new states:",  
              sorted(new))
```

```
        frontier = new
```

```
    print("Total states found:", len(Seen))
```

[Output]

level 1 → 2 new states: [(0,3), (4,0)]

level 2 → 3 new states: [(1,3), (3,0), (4,3)]

level 3 → 2 new states: [(1,0), (3,3)]

level 4 → 2 new states: [(0,1), (4,2)]

level 5 → 2 new states: [(0,2), (4,1)]

*** GOAL: 2L in X at level 6 ***

*** GOAL: 2L in X at level 6 ***

level 6 $\rightarrow$ 2 new states: $[(2,0), (2,3)]$

level 7 $\rightarrow$ 0 new states: $[\]$

Total states found: 14